

Os guardiões de Eldoria

1. Itens iniciais

Objetivos

- Aplicar os fundamentos da programação orientada a objetos em Java, por meio da criação de classes, objetos, herança, polimorfismo e composição, para estruturar soluções modulares e reutilizáveis.
- Aprofundar os conceitos de herança, visibilidade, classes abstratas e polimorfismo dinâmico na linguagem Java para estruturar programas orientados a objetos que promovam reutilização e flexibilidade no código.
- Aplicar estruturas de dados da linguagem Java, por meio da Java Collections Framework e API de Streams, para organizar, manipular e otimizar conjuntos de objetos de forma eficiente.

Introdução

Olá! Boas-vindas ao desafio de programação! A TechNova, uma empresa de tecnologia, contratou você para restaurar a ordem e a lógica no desenvolvimento do jogo de RPG Os guardiões de Eldoria. Sua missão é aplicar os princípios da programação orientada a objetos com Java para otimizar as funcionalidades do jogo.

No mercado de trabalho em TI, o **domínio da programação** orientada a objetos ajuda a criação de sistemas escaláveis, reutilizáveis e organizados. Seja em uma startup, consultoria ou grande empresa de tecnologia, pessoas desenvolvedoras são desafiadas todos os dias a aplicar conceitos como herança, polimorfismo, encapsulamento e estruturas de dados para **construir soluções** robustas e eficientes.

Diante isso, este desafio simula um cenário real de desenvolvimento, em que cada etapa reflete habilidades exigidas pelo mercado — desde o design de classes até o gerenciamento de grandes volumes de objetos.

Assista ao vídeo e inicie sua jornada na POO em Java! Embarque na missão de restaurar a ordem no reino digital de Eldoria e dominar a programação orientada a objetos com Java.

> Conteúdo interativo
>
> Acesse a versão digital para assistir ao vídeo.

Antes de navegar e avançar pelos níveis do desafio, você conhecerá o cenário em que tudo acontece e descobrirá qual será a sua missão. Prepare-se!

Cenário

Eldoria é um mundo dividido entre vilas, guildas e arenas — um reino digital que precisa com urgência de estrutura. No passado, seus sistemas eram organizados, seus dados bem agrupados e os programas eram eficientes. Mas a negligência e a falta de manutenção lançaram a terra no caos: personagens sem atributos, métodos desatualizados, listas mal implementadas e sistemas sem coesão.

Diante disso, você foi nomeado programador-líder para reconstruir as bases desse mundo com Java, utilizando os princípios da orientação a objetos e os recursos avançados da linguagem. A cada fase, você enfrentará problemas típicos do desenvolvimento profissional: projetar classes reutilizáveis, aplicar abstrações, manipular grandes coleções de dados e organizar o código com clareza e desempenho.

Sua missão

Você deve desenvolver um sistema modular inspirado em um universo medieval de fantasia, no qual os personagens (guerreiros, magos e curandeiros) se organizam em grupos, participam de batalhas e interagem entre si. Essa estrutura simula, na prática, a arquitetura de sistemas orientados a objetos e a complexidade de suas relações.

Para isso, você deverá desenvolver um minigame em Java ambientado no reino fictício de Eldoria, em que cada etapa representa a evolução do programador no domínio da orientação a objetos. A jornada começa com a criação de personagens, passa pela implementação de hierarquias de classes com herança e polimorfismo e culmina no agrupamento e na manipulação eficiente de dados usando as coleções da linguagem.

Cada fase trará **desafios crescentes** que exigirão entendimento conceitual, aplicação prática e organização de código para modelar um sistema coeso, escalável e alinhado às necessidades do mundo real do desenvolvimento de software.

O programa em Java deverá:

- Criar personagens com atributos e métodos encapsulados, promovendo boas práticas de reúso e coesão.
- Utilizar herança e polimorfismo para estruturar subclasses específicas, com comportamentos distintos e reutilização de código.
- Manipular grupos de personagens com coleções, simulando batalhas, interações e estratégias entre diferentes tipos de heróis.
- Organizar o código em múltiplos arquivos, promovendo modularidade e clareza na arquitetura.
- Aplicar técnicas de ordenação, comparação e agrupamento de dados usando recursos modernos da linguagem Java.

Este é um desafio técnico e criativo: seu código será sua arma, e sua organização, sua estratégia.

A jornada começa agora, guardião. Vamos lá!

Ambiente Java: ferramentas e primeiro código

Descubra neste vídeo o que são JDK, JRE e JVM e aprenda a instalar e configurar o ambiente de desenvolvimento. Explore também IDEs como Eclipse e IntelliJ, ajuste variáveis de ambiente e comece a escrever seu primeiro programa Java!

Conteúdo interativo

Acesse a versão digital para assistir ao vídeo.

Conceito da linguagem Java

Criada em 1995 pela Sun Microsystems, Java é uma linguagem de programação bastante utilizada no desenvolvimento de sistemas desktop, web e mobile. Hoje é mantida pela Oracle Corporation. Um de seus principais diferenciais é a **portabilidade**, graças à Java Virtual Machine (JVM), que permite executar o mesmo programa em diferentes sistemas operacionais. Para iniciarmos na linguagem, precisaremos compreender a estrutura do ambiente Java e configurar com exatidão as ferramentas de desenvolvimento.

O que é Java (JDK, JRE)

O ecossistema Java é composto por dois elementos fundamentais:

JDK (Java Development Kit)		JRE (Java Runtime Environment)	
Reúne as ferramentas necessárias para o desenvolvimento de programas Java, incluindo compilador (javac), bibliotecas padrão e utilitários.	**Resumo:** cria, compila e executa aplicações Java (inclui o JRE).	É destinado à execução de programas já compilados, oferecendo apenas a JVM e bibliotecas de tempo de execução.	**Resumo:** executa aplicações Java.

Lembre-se que, para o desenvolvimento, o **JDK** é indispensável!

Instalação do JDK (Windows e Linux)

No Windows, a instalação envolve o download do JDK do site da Oracle. Após aceitar os termos de licença, basta escolher a versão apropriada e seguir o instalador.

No Linux, é possível instalar o OpenJDK via terminal. Para distribuições Debian/Ubuntu:

```
sudo apt update
sudo apt install openjdk-17-jdk
```

Após a instalação, o comando java -version deve exibir a versão instalada.

Instalação e configuração do IDE (Eclipse, VS Code ou IntelliJ)

Vamos entender primeiro que IDEs (ambientes de desenvolvimento integrado) facilitam a escrita, compilação e execução de programas Java. Os mais comuns são:

Eclipse
É robusto, gratuito e bastante utilizado.

IntelliJ IDEA
Oferece versões Community (gratuita) e Ultimate (comercial).

Visual Studio Code
É um editor leve, com suporte a Java via extensões.

Para instalar o Eclipse, acesse a seção de downloads no site da Eclipse e selecione a opção Eclipse IDE for Java Developers. No caso do IntelliJ ou VS Code, a instalação também é simples e intuitiva. Após instalar o IDE escolhido, certifique-se de que ele reconheceu, de maneira certa, o JDK, configurando o caminho nas preferências, se necessário.

Configuração de variáveis de ambiente

Em sistemas Windows, a configuração das variáveis de ambiente permite a execução dos comandos java e javac via terminal.

Confira a seguir o passo a passo:

1. Vá em: Painel de Controle > Sistema > Configurações Avançadas do Sistema > Variáveis de Ambiente.
2. Crie a variável JAVA_HOME apontando para o diretório de instalação do JDK. Por exemplo: C:\Program Files\Java\jdk-17
3. Adicione no campo Path: %JAVA_HOME%\bin

No Linux, adicione no arquivo ~/.bashrc ou ~/.zshrc:

```
export JAVA_HOME=/usr/lib/jvm/java-17-openjdk-amd64
export PATH=$JAVA_HOME/bin:$PATH
```

Após salvar, execute source ~/.bashrc para aplicar.

Primeiro programa Java: estrutura básica (HelloWorld.java)

Com o ambiente pronto, é hora de escrever seu primeiro programa. Vamos criar o famoso Hello, World! para verificar se tudo está funcionando como o esperado.

Exemplo:

```java
public class HelloWorld {
    public static void main(String[] args) {
        System.out.println("Hello, World!");
    }
}
```

O programa define uma classe chamada HelloWorld, que contém o método main. Esse método é o ponto de entrada do programa Java, ou seja, é onde a execução começa. O comando System.out.println é usado para imprimir a mensagem 'Hello, World!' no console.

Compilação e execução via terminal e via IDE

Se você estiver utilizando o terminal, siga estes passos:

1. Compile o programa: javac HelloWorld.java
2. Execute: java HelloWorld

Você verá no console:

Hello, World!

Porém, se estiver utilizando uma IDE, basta clicar com o botão direito sobre o arquivo HelloWorld.java e escolher “Run” (Executar). A IDE cuidará, de modo automático, da compilação e execução.

Paradigma orientado a objetos em Java

Assista ao vídeo para saber como transformar problemas reais em soluções de software com a programação orientada a objetos em Java. Conheça os pilares do paradigma orientados a objetos, como classes, objetos, atributos e métodos. Veja também exemplos práticos que mostram como encapsular dados e comportamentos de forma clara, reutilizável e escalável. Não perca!

Conteúdo interativo

Acesse a versão digital para assistir ao vídeo.

Conceito de paradigmas orientados a objetos



Você já se perguntou por que pessoas programadoras experientes conseguem transformar problemas do mundo real em soluções robustas, escaláveis e reutilizáveis? O segredo está no **modo** como organizam o código, o qual chamamos: paradigma orientado a objetos.

Ele vai muito além da codificação, pois é uma forma de pensar, projetar e construir software inspirada no mundo real, em que os elementos do sistema são representados por objetos que interagem entre si.
Portanto, ao dominar esses conceitos, você se posiciona como alguém que escreve código e modela soluções.

De procedural para orientado a objetos

Antes de entrar na orientação a objetos (OO), é importante entender de onde ela veio. Na programação procedural, o foco está em funções e procedimentos: o código é escrito em sequência, e os dados são processados como entradas e saídas dessas funções. Para pequenos scripts, essa abordagem funciona bem, mas falha à medida que o sistema cresce.

Na OO, o foco está nos objetos que são pequenas unidades que encapsulam dados (atributos) e comportamentos (métodos). Em vez de processar os dados separadamente, os próprios objetos sabem como agir.

Veja agora o quadro com as abordagens estudadas:

Aspecto	Programação procedural	Programação orientada a objetos
Organização	Funções e variáveis soltas	Classes e objetos interconectados
Reutilização	Baixa	Alta (herança, composição)
Manutenção	Mais difícil em sistemas grandes	Mais fácil com separação de responsabilidades
Escalabilidade	Limitada	Alta, com encapsulamento e modularização
Exemplos	C, Pascal	Java, Python (com OO), C#

Tabela: Comparação entre programação procedural e programação orientada a objetos.
Curadoria de TI.

Elementos fundamentais da orientação a objetos

Dominar OO em Java significa entender com clareza os seguintes conceitos:

Classe

É como um molde ou projeto. Define o que os objetos daquele tipo sabem (atributos) e o que sabem fazer (métodos).

Objeto

É a instância concreta de uma classe. Quando você usa **new** em Java, está criando um objeto.

Atributo

É uma variável pertencente à classe ou ao objeto. Representa características.

Método

É uma função associada à classe. Representa ações ou comportamentos.

Instância

É o termo técnico dado a um objeto específico criado a partir de uma classe.

Vamos pensar em uma classe que representa um usuário de sistema com nome, e-mail e senha. Ele também pode fazer login.

Arquivo: Usuario.java

```
java
public class Usuario {
    String nome;
    String email;
    String senha;

    public void fazerLogin() {
        System.out.println(nome + " está tentando fazer login.");
    }
}
```

Agora instanciamos um objeto do tipo Usuario.

Arquivo: Main.java

```java
public class Main {
    public static void main(String[] args) {
        Usuario u1 = new Usuario();
        u1.nome = "Fulano";
        u1.email = "fulano@email.com";
        u1.senha = "123456";

        u1.fazerLogin();  // Saída: Fernanda está tentando fazer login.
    }
}
```

Perceba como o objeto u1 tem seus próprios dados e sabe executar o método fazerLogin(). Esse encapsulamento de dados + comportamento é a **base** da orientação a objetos.

O ciclo de vida de um objeto em Java

Entender como um objeto “nasce”, “vive” e “morre” no programa possibilita trabalhar com eficiência. Veja!

1. A classe é definida com atributos e métodos.
2. O objeto é instanciado com **new**.
3. Os atributos são definidos.
4. Os métodos são chamados.
5. Ao final da execução, o objeto é descartado da memória pela JVM.

Criando e usando classes e objetos em Java

Aprenda neste vídeo a transformar teoria em prática com a criação de classes e objetos em Java. Veja como declarar atributos, métodos, construtores e utilizar o operador ponto para acessar comportamentos, com exemplos práticos.



Conteúdo interativo

Acesse a versão digital para assistir ao vídeo.

Conceitos iniciais de construção de classes e objetos

Chegou o momento de transformar teoria em código! Em Java, a construção de classes é o alicerce de toda a estrutura de um sistema orientado a objetos. Afinal, é a partir delas que os objetos ganham forma, dados e comportamento — e é por isso que a clareza na definição dessas estruturas é fundamental.

Sintaxe de declaração de classes

Em Java, uma classe é declarada com a palavra-chave **class**, seguida do nome da classe. Os nomes devem começar com letra maiúscula e seguir a convenção **PascalCase**.

PascalCase

Estilo de escrita em que cada palavra começa com letra maiúscula, sem espaços ou símbolos.

Exemplo: Classe Produto

```java
public class Produto {
    String nome;
    double preco;

    public void exibirInformacoes() {
        System.out.println("Produto: " + nome + " | Preço: R$" + preco);
    }
}
```

Note que essa classe possui dois atributos (nome, preco) e um método (exibirInformacoes), responsável por mostrar os dados no console.

Atributos e seus tipos

Vamos entender que atributos são variáveis internas da classe. Eles podem ser de dois tipos principais:

Primitivo	De referência
int, double, float, boolean, char	String, arrays, outras classes
Exemplo: int quantidade;	Exemplo: String nomeProduto;

Métodos: ações dos objetos

Representam o comportamento dos objetos. Sua sintaxe inclui um tipo de retorno, nome e lista de parâmetros (se houver).

Exemplo:

```java
public void aumentarPreco(double percentual) {
    preco = preco + (preco * percentual / 100);
}
```

No exemplo, vimos que o método aumenta o preço do produto com base em um percentual informado.

Construtores: o nascimento do objeto

Se formos definir um construtor, diremos que ele é um método especial utilizado no momento da criação do objeto. Em Java, ele tem o mesmo nome da classe e não possui tipo de retorno.

Exemplo:

```java
public class Produto {
    String nome;
    double preco;

    public Produto(String nome, double preco) {
        this.nome = nome;
        this.preco = preco;
    }
}
```

Agora podemos criar objetos com dados definidos. Veja:

```
Produto p1 = new Produto("Notebook", 3500.00);
```

Dica: Quando usamos this.nome, estamos nos referindo ao atributo da classe, diferenciando-o do parâmetro do método.

Sobrecarga de construtores

Java permite criar múltiplos construtores com diferentes assinaturas. Isso é útil para dar flexibilidade à criação dos objetos.

Exemplo:

```java
public Produto(String nome) {
    this.nome = nome;
    this.preco = 0.0;
}
```

Agora é possível instanciar um **Produto** apenas com o nome, utilizando o construtor correspondente com o operador **new** que iremos detalhar a seguir.

```
Produto p2 = new Produto("Monitor");
```

Instanciando objetos com new

A criação de um objeto em Java é feita com o operador **new**, que aloca memória e chama o construtor da classe.

Exemplo:

```
Produto p = new Produto("Mouse", 99.90);
```

Esse comando cria um objeto do tipo Produto com nome e preço definidos.

Acesso via operador ponto (.)

Depois de instanciar o objeto, basta usar a variável p com o operador ponto (.) para acessar seus atributos ou chamar métodos.

Exemplo:

```
p.nome = "Teclado";
p.aumentarPreco(10);
p.exibirInformacoes();
```

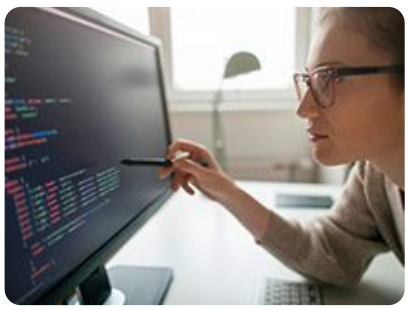
Aqui, estamos modificando o nome do produto, aplicando aumento de 10% e exibindo as informações.

Encapsulamento: protegendo e organizando os dados das suas classes

Descubra neste vídeo como o encapsulamento em Java protege os dados das suas classes, evitando alterações indesejadas e garantindo segurança. Aprenda também a usar modificadores de acesso, getters e setters corretamente.

> Conteúdo interativo
>
> Acesse a versão digital para assistir ao vídeo.

Conceito de encapsulamento



Você já deve ter ouvido que “não se deve deixar as portas abertas”. Essa metáfora se aplica ao código orientado a objetos. Quando criamos classes com atributos visíveis e modificáveis por qualquer parte do sistema, abrimos portas para problemas difíceis de rastrear, como alterações indesejadas de valores ou comportamentos incoerentes.

O encapsulamento é uma prática que consiste em proteger os atributos internos de uma classe, controlando seu acesso por meio de métodos próprios. É uma das bases da orientação a objetos e uma exigência constante em projetos reais.

Modificadores de acesso: controlando quem vê o quê

Em Java, podemos definir o nível de acesso de uma classe, atributo ou método usando determinados modificadores. A seguir, veja como o acesso é permitido de acordo com o modificador:

public

Acesso permitido por qualquer classe de qualquer pacote.

private

Acesso permitido só dentro da própria classe.

protected

Acesso permitido dentro da mesma classe, subclasses ou mesmo pacote.

(sem nada)

Acesso permitido apenas dentro do mesmo pacote.

Ao tornar um atributo private, você impede que outras classes o modifiquem diretamente. Assim, qualquer mudança precisa passar por métodos específicos (getters e setters), que você controla.

Exemplo com encapsulamento:

```java
public class ContaBancaria {
    private double saldo;

    public double getSaldo() {
        return saldo;
    }

    public void depositar(double valor) {
        if (valor > 0) {
            saldo += valor;
        }
    }

    public void sacar(double valor) {
        if (valor > 0 && valor <= saldo) {
            saldo -= valor;
        }
    }
}
```

Perceba que saldo é um dado sensível. Ele não deve ser acessado de maneira direta. Por isso:

- getSaldo() permite consultar o valor.
- depositar() e sacar() impõem regras para alterar o saldo, garantindo segurança e integridade.

Em sistemas reais, dados mal protegidos podem gerar falhas de negócio, inconsistências em banco de dados e brechas de segurança. Assim, ao encapsular de maneira correta, você define as fronteiras de cada classe e evita efeitos colaterais indesejados.

Boas práticas de encapsulamento

Para aplicar o encapsulamento de forma eficaz e tornar seu código seguro e organizado, siga estas diretrizes fundamentais:

- Declare atributos como private sempre que possível.
- Exponha apenas os métodos, de fato, necessários (ou seja, use o modificador public com **parcimônia**).
- Use getters e setters apenas quando fizer sentido para o domínio da aplicação.
- Valide os dados dentro dos métodos de modificação (ex: evitar valores negativos).

Quando usar getter e setter

Nem sempre você precisa expor um atributo com get e set. Em alguns casos, pode ser mais adequado disponibilizar apenas métodos de ação. Compare a seguir!

a) Exemplo de um caso em que seria adequado:

```
java

public void ativarConta() {
    this.ativa = true;
}
```

b) Exemplo de um caso em que seria desnecessário:

```
java

public void setAtiva(boolean ativa) {
    this.ativa = ativa;
}
```

No último exemplo, qualquer parte do código poderia ativar ou desativar a conta sem nenhum controle. Isso pode ser perigoso em sistemas de negócio.

Hora de codar

Assista ao vídeo e aprenda, na prática, a criar classes, atributos, métodos e objetos em Java.



Conteúdo interativo

Acesse a versão digital para assistir ao vídeo.

Vamos praticar? Para fixar os conceitos de classe, atributos, métodos e instanciação de objetos, você irá criar uma classe Produto com os seguintes atributos:

- nome (String)

- preco (double)
- quantidadeEmEstoque (int)

Etapas do desenvolvimento

Implemente:

1. Um construtor que inicializa todos os atributos.
2. Um método exibirInformacoes() que imprime as informações do produto no console.

Em seguida, instancie dois produtos e chame o método exibirInformacoes() para cada um.



Dica

Use o operador ponto para acessar os atributos e métodos dos objetos.

Desafio: nível novato

Assista ao vídeo e explore, na prática, os pilares da orientação a objetos ao dar vida a personagens que representam o coração do paradigma em Java.



Conteúdo interativo

Acesse a versão digital para assistir ao vídeo.

O que você vai fazer

Você vai criar uma classe chamada Personagem com atributos que representem características básicas de um herói. Em seguida, irá instanciar dois objetos dessa classe, atribuindo valores. Por fim, você vai imprimir as informações dos personagens usando System.out.println. Esse é o primeiro passo para dominar o universo de objetos que interagem entre si — o coração da orientação a objetos.

Requisitos funcionais

Em seu programa, você deverá criar uma classe chamada Personagem.

A classe deve conter os seguintes atributos:

- nome (String)
- classe (String): exemplo de valores possíveis — Mago, Guerreiro, Arqueiro.

- nivel (int)
- pontosDeVida (int)
- poderBase (double)

A classe deve ter um método exibirStatus(), que imprime no console todas as informações do personagem com formatação legível.

No método main, você deverá:

1. Instanciar dois personagens (utilizando new).
2. Atribuir valores diretamente aos atributos.
3. Chamar o método exibirStatus() para cada personagem.

Exemplo de saída esperada:

```
Nome: Arthemis
Classe: Arqueira
Nível: 5
Pontos de Vida: 80
Poder Base: 12.5
```

Requisitos não funcionais

Observe os seguintes elementos importantes:

- Não utilizar estruturas de decisão (if, switch) ou repetição (for, while, do-while).
- Todo o código deve estar organizado, com identação adequada e nomes de variáveis significativos.
- O projeto deve conter comentários simples (//), explicando partes importantes do código.
- O código deve compilar e executar sem erros.
- Utilize, de preferência, IntelliJ, Eclipse ou VS Code com extensão Java.

Acessando o desafio

Acesse o GitHub Classroom. Nesse ambiente, você terá acesso ao repositório padrão do desafio. Caso ainda não tenha uma conta no GitHub, não se preocupe: você pode criar uma grátis, clicando no link.

Herança – aspectos elementares de herança e visibilidade

Neste vídeo, explicamos como a herança em Java permite reutilizar atributos e métodos entre classes, tornando seu código mais limpo e escalável. Você vai aprender a construir hierarquias, aplicar visibilidade adequada e usar o construtor super para promover organização e evitar redundâncias em sistemas orientados a objetos. Não perca!

> Conteúdo interativo
>
> Acesse a versão digital para assistir ao vídeo.

Conceito de herança

Um dos principais fundamentos da programação orientada a objetos (POO) é a herança, que atua na construção de sistemas mais organizados, reutilizáveis e de fácil manutenção. Em Java, a herança é implementada por meio da palavra-chave **extends** e permite que uma classe herde atributos e métodos de outra. Essa capacidade de especializar ou reaproveitar comportamentos permite a escalabilidade de sistemas complexos.

Imagine um cenário em que você esteja desenvolvendo um sistema de gerenciamento de pessoas dentro de uma organização. Muitas entidades que você irá modelar compartilham características em comum, como nome, CPF ou data de nascimento. Assim, em vez de duplicar esses atributos em cada classe, você pode criar uma classe mais genérica, chamada Pessoa, e fazer com que outras classes herdem suas propriedades. Isso evita redundância e fortalece a coesão entre os elementos do sistema.



Considere o seguinte exemplo:

```java
public class Pessoa {
    private String nome;
    private String cpf;

    public Pessoa(String nome, String cpf) {
        this.nome = nome;
        this.cpf = cpf;
    }

    public String getNome() {
        return nome;
    }

    public String getCpf() {
        return cpf;
    }
}
```

Essa é a classe base. A seguir, criaremos uma subclasse Aluno, que herda os atributos nome e CPF e adiciona um novo atributo específico: matrícula. Veja!

```java
public class Aluno extends Pessoa {
    private String matricula;

    public Aluno(String nome, String cpf, String matricula) {
        super(nome, cpf);
        this.matricula = matricula;
    }

    public String getMatricula() {
        return matricula;
    }
}
```

A palavra-chave **super** é utilizada para chamar o construtor da superclasse (Pessoa). Isso permite que os atributos herdados sejam, de fato, inicializados.

Ao utilizar esse modelo, você pode, por exemplo, instanciar objetos do tipo Aluno e acessar seus dados de forma organizada:

```
Aluno aluno = new Aluno("João Silva", "123.456.789-00", "2021001");

System.out.println("Nome: " + aluno.getNome());

System.out.println("CPF: " + aluno.getCpf());

System.out.println("Matrícula: " + aluno.getMatricula());
```

Esse modelo de reutilização de código é um exemplo claro de como a herança contribui para o desenvolvimento de sistemas mais limpos, nos quais o foco é na manutenção e evolução do código.

Herança e visibilidade

Os modificadores de acesso ajudam a entender a herança em Java. Eles determinam o nível de visibilidade dos atributos e métodos de uma classe e influenciam o que é ou não herdado pelas subclasses.

Os três principais modificadores são:

public

O membro é acessível de qualquer parte do código, inclusive por outras classes em outros pacotes.

protected

O membro é acessível pelas subclasses, mesmo que estejam em pacotes diferentes.

private

O membro é acessível apenas dentro da própria classe em que foi declarado. Ou seja, não é herdado pelas subclasses.

Considere este exemplo:

```java
public class Pessoa {
    protected String nome;
    private String cpf;

    public Pessoa(String nome, String cpf) {
        this.nome = nome;
        this.cpf = cpf;
    }

    public String getNome() {
        return nome;
    }
}

public class Funcionario extends Pessoa {
    private double salario;

    public Funcionario(String nome, String cpf, double salario) {
        super(nome, cpf);
        this.salario = salario;
    }

    public void mostrarDados() {
        System.out.println("Nome: " + nome); // Acesso permitido (protected)
        // System.out.println("CPF: " + cpf); // Erro: cpf é private em Pessoa
    }
}
```

No caso visto, a variável nome é acessada dentro da subclasse Funcionario, porque foi declarada como protected. Já o CPF, mesmo sendo parte da superclasse, não pode ser acessado, pois está como private. Isso demonstra a importância de compreender a visibilidade ao projetar hierarquias.

Recomendações práticas:

- Sempre que possível, mantenha os atributos como private e ofereça acesso por meio de getters e setters. Isso promove encapsulamento, outro pilar da POO.
- Use protected com cautela, sobretudo, em projetos maiores, pois pode dificultar o rastreamento de dependências entre classes.
- Use public apenas quando a visibilidade ampla for, de fato, necessária.

Hierarquia de herança

Em Java, uma classe pode estender apenas outra classe (herança simples). Isso evita ambiguidade e torna o modelo de herança mais previsível. Ainda assim, você pode criar estruturas hierárquicas complexas com vários níveis. Veja!

```
public class Pessoa {}

public class Funcionario extends Pessoa {}

public class Gerente extends Funcionario {}
```

Com a cadeia vista, Gerente herda tanto os atributos e métodos de Funcionario quanto os de Pessoa, promovendo reutilização e especialização.

Por fim, entender de modo correto herança em Java contribui para o desenvolvimento de sistemas modulares, reutilizáveis e escaláveis. Mais do que um recurso de linguagem, trata-se de uma estratégia de projeto que aproxima o código da realidade, promovendo legibilidade, manutenção e expansão futuras.

Exemplos de herança em Java

Compreenda neste vídeo como a herança em Java ajuda a criar sistemas organizados e reutilizáveis. A partir de cenários reais, como locadoras de veículos e lojas virtuais, aprenda a estruturar superclasses e especializar comportamentos, promovendo clareza, escalabilidade e manutenção eficiente no desenvolvimento orientado a objetos.

> Conteúdo interativo
>
> Acesse a versão digital para assistir ao vídeo.

Vamos agora exercitar os conceitos de herança! Cada cenário foi escolhido para refletir problemas comuns no desenvolvimento de sistemas, em que a herança pode trazer clareza, reutilização de código e estrutura mais coesa.

Veículos em uma locadora



Imagine que você está desenvolvendo um sistema para uma locadora de veículos. Eles possuem dados em comum (como marca, modelo e placa), mas também características específicas conforme o tipo (carro, moto ou caminhão).

A classe base Veiculo armazena os dados que todos os veículos compartilham. Note que o atributo placa é privado, e os demais são protegidos para permitir acesso controlado pelas subclasses. Além disso, o método getDescricao() pode ser reaproveitado por todas as subclasses. Confira!

Classe Base:

```java
public class Veiculo {
    protected String marca;
    protected String modelo;
    private String placa;

    public Veiculo(String marca, String modelo, String placa) {
        this.marca = marca;
        this.modelo = modelo;
        this.placa = placa;
    }

    public String getDescricao() {
        return marca + " " + modelo + " - Placa: " + placa;
    }
}
```

A subclasse Carro herda os atributos da classe Veiculo e adiciona novo atributo numeroPortas. A palavra-chave super é usada para chamar o construtor da superclasse.

Subclasse Carro:

```java
public class Carro extends Veiculo {
    private int numeroPortas;

    public Carro(String marca, String modelo, String placa, int numeroPortas) {
        super(marca, modelo, placa);
        this.numeroPortas = numeroPortas;
    }

    public int getNumeroPortas() {
        return numeroPortas;
    }
}
```

De forma semelhante, a subclasse Moto adiciona um atributo booleano indicando se possui baú e aproveita o construtor da superclasse.

Subclasse Moto:

```java
public class Moto extends Veiculo {
    private boolean possuiBaú;

    public Moto(String marca, String modelo, String placa, boolean possuiBaú) {
        super(marca, modelo, placa);
        this.possuiBaú = possuiBaú;
    }

    public boolean isPossuiBaú() {
        return possuiBaú;
    }
}
```

Uso: classe de execução com o método main:

```java
public class TesteLocadora {
    public static void main(String[] args) {

        Carro carro = new Carro("Fiat", "Argo", "ABC-1234", 4);
        Moto moto = new Moto("Honda", "CG 160", "XYZ-5678", true);

        System.out.println(carro.getDescricao());
        System.out.println("Portas: " + carro.getNumeroPortas());

        System.out.println(moto.getDescricao());
        System.out.println("Possui baú: " + moto.isPossuiBaú());
    }
}
```

Esse exemplo ilustra como um sistema pode ser construído com classes reutilizáveis e especializadas, o que facilita a manutenção e a leitura do código.

Produtos em um e-commerce

Vamos modelar um sistema para um e-commerce com produtos variados. A classe base Produto contém os atributos nome e preço, que são comuns a todos os produtos.

Classe Produto:

```java
public class Produto {
    protected String nome;
    protected double preco;

    public Produto(String nome, double preco) {
        this.nome = nome;
        this.preco = preco;
    }

    public String getInfo() {
        return nome + " - R$" + preco;
    }
}
```

A subclasse Livro herda de Produto e acrescenta autor e ISBN. O método getDetalhes() reaproveita o método da superclasse e adiciona as novas informações.

Subclasse Livro:

```java
public class Livro extends Produto {
    private String autor;
    private String isbn;

    public Livro(String nome, double preco, String autor, String isbn) {
        super(nome, preco);
        this.autor = autor;
        this.isbn = isbn;
    }

    public String getDetalhes() {
        return getInfo() + " | Autor: " + autor + " | ISBN: " + isbn;
    }
}
```

A subclasse Eletronico adiciona o fabricante e o tempo de garantia. Note o uso da herança e o reaproveitamento do método da superclasse!

Subclasse Eletronico:

```java
public class Eletronico extends Produto {
    private String fabricante;
    private int garantiaMeses;

    public Eletronico(String nome, double preco, String fabricante, int garantiaMeses) {
        super(nome, preco);
        this.fabricante = fabricante;
        this.garantiaMeses = garantiaMeses;
    }

    public String getDetalhes() {
        return getInfo() + " | Fabricante: " + fabricante + " | Garantia: " +
garantiaMeses + " meses";
    }
}
```

Uso: classe de execução com o método main:

```java
public class TesteEcommerce {
    public static void main(String[] args) {

        Livro livro = new Livro("Clean Code", 89.90, "Robert C. Martin", "9780132350884");
        Eletronico celular = new Eletronico("Smartphone X", 1999.00, "TechCorp", 12);

        System.out.println(livro.getDetalhes());
        System.out.println(celular.getDetalhes());
    }
}
```

O exemplo evidencia o uso de herança para especializar objetos conforme o domínio do problema, mantendo uma estrutura comum que permite reaproveitamento e extensibilidade.

A saída esperada é:

```
Clean Code - R$89.9 | Autor: Robert C. Martin | ISBN: 9780132350884

Smartphone X - R$1999.0 | Fabricante: TechCorp | Garantia: 12 meses
```

Heróis de Eldoria: estruturando os tipos de personagens

Vamos modelar a estrutura básica dos heróis do nosso jogo. Todos os personagens possuem características em comum, como nome e nível. A partir dessa base, criaremos classes mais específicas, como Guerreiros e Magos, que adicionam seus próprios atributos.

Classe Personagem

A classe base Personagem armazena os dados que todos os heróis compartilham. Ela também possui um método para retornar uma descrição básica, que será reutilizado pelas subclasses.

```java
public class Personagem {
    protected String nome;
    protected int nivel;

    public Personagem(String nome, int nivel) {
        this.nome = nome;
        this.nivel = nivel;
    }
    //Método base que retorna a informação comum a todos os personagens
    public String getDescricaoBase() {
        return "Nome: " + nome + " | Nível: " + nivel;
    }
}
```

Subclasse Guerreiro

Herda as características da classe Personagem e adiciona um novo atributo específico: forca. Ela também possui um método que **reutiliza** a descrição da classe pai e **adiciona** sua própria informação.

```java
public class Guerreiro extends Personagem {

    private int forca;

    public Guerreiro(String nome, int nivel, int forca) {
        super(nome, nivel);    // Chama o construtor da classe Personagem
        this.forca = forca;

    }
    // Este método utiliza o método da classe pai para construir sua descrição
    public String exibirDescricaoCompleta() {
        // Reutiliza getDescricaoBase() e adiciona a informação específica
        return getDescricaoBase() + " | Força: " + forca;
    }
}
```

Subclasse Mago

Herda de Personagem e adiciona o atributo inteligencia, criando um método de descrição que também aproveita a estrutura da classe pai. Veja!

```java
public class Mago extends Personagem {
    private int inteligencia;
    public Mago(String nome, int nivel, int inteligencia) {
        super(nome, nivel); // Chama o construtor da classe Personagem
        this.inteligencia = inteligencia;
    }
    // Este método também reutiliza o método da classe pai
    public String exibirDescricaoCompleta() {
        return getDescricaoBase() + " | Inteligência: " + inteligencia;
    }
}
public class Arena {
    public static void main(String[] args) {
        Guerreiro guerreiro = new Guerreiro("Arthus", 10, 85);
        Mago mago = new Mago("Elenara", 12, 95);

        // Exibe as informações completas de cada herói
        System.out.println(guerreiro.exibirDescricaoCompleta());
        System.out.println(mago.exibirDescricaoCompleta());
    }
}
```

Saída esperada pare este exemplo:

```
Nome: Arthus | Nível: 10 | Força: 85
Nome: Elenara | Nível: 12 | Inteligência: 95
```

Passos para execução e teste dos exemplos

Para organizar projetos em Java em ambientes de desenvolvimento (IDEs), como Eclipse, IntelliJ ou VS Code, confira o passo a passo:

1. Crie um arquivo para cada classe de modelo:

[NomeDaClasseBase].java

[NomeDaSubclasse1].java

[NomeDaSubclasse2].java

2. Crie um arquivo final para a execução, por exemplo, Teste.java.

3. Crie o método main e cole o código do bloco **"Uso:"** dentro de Teste.java.

4. Compile e execute o arquivo Teste.java.

A aplicação prática da herança permite que sistemas sejam construídos com maior organização e reaproveitamento de código. A definição clara de superclasses e subclasses evita redundância e facilita

futuras manutenções. Assim, ao estruturar seu código com hierarquias bem definidas, você se aproxima das boas práticas da orientação a objetos, um diferencial importante para atuação em projetos reais de software. Ao compreender e exercitar esses exemplos, você está mais preparado para modelar soluções robustas e alinhadas às necessidades do mercado.

Polimorfismo em Java

Assista ao vídeo e entenda como o polimorfismo em Java torna seu código mais flexível, reutilizável e preparado para crescer. Você vai aprender, com exemplos práticos, a manipular objetos de diferentes classes de forma uniforme, dominar o dynamic dispatch e aplicar esse conceito essencial da orientação a objetos em sistemas extensíveis e robustos. Confira!

> Conteúdo interativo
>
> Acesse a versão digital para assistir ao vídeo.

Conceito de polimorfismo em Java

Polimorfismo tem origem no grego e significa "muitas formas". Em Java, isso se traduz na habilidade de tratar objetos de diferentes subclasses por meio da referência de sua superclasse comum. Isso permite que, mesmo que diferentes objetos implementem comportamentos distintos, eles possam ser manipulados de maneira uniforme e previsível. Vamos explorar mais!

Polimorfismo de tempo de execução

É um dos tipos mais utilizados em Java. Ocorre quando uma subclasse fornece uma implementação específica de um método que já está definido na sua superclasse. Essa sobrescrita permite que, ao utilizar uma referência da superclasse, o método correto seja invocado de acordo com o tipo real do objeto.

Veja o seguinte exemplo:

```java
public class Animal {
    public void fazerSom() {
        System.out.println("Som genérico de animal");
    }
}

public class Cachorro extends Animal {
    @Override
    public void fazerSom() {
        System.out.println("Latido!");
    }
}

public class Gato extends Animal {
    @Override
    public void fazerSom() {
        System.out.println("Miado!");
    }
}
```

Nesse caso, cada subclasse (Cachorro e Gato) sobrescreve o método fazerSom da classe Animal. Se usarmos um vetor com referência à classe Animal, teremos a seguinte chamada:

```java
Animal[] animais = { new Cachorro(), new Gato(), new Cachorro() };

for (Animal a : animais) {
    a.fazerSom();
}
```

A saída será:

```
Latido!
Miado!
Latido!
```

Mesmo que as variáveis a sejam do tipo Animal, o método executado é o da subclasse correspondente ao objeto instanciado. Isso é o que torna o código extensível: podemos adicionar novas subclasses sem alterar o código que manipula a lista de Animal.

Variáveis de referência X Tipo do objeto

Um dos pontos mais importantes na compreensão do polimorfismo é saber diferenciar o tipo da variável de referência e o tipo real do objeto:

Tipo da variável de referência		Tipo real do objeto
Determina quais métodos podem ser chamados de modo direto.	⊗	Determina qual implementação do método será executada em tempo de execução.

Considere o seguinte código:

```
Animal a1 = new Cachorro();
a1.fazerSom(); // Executa o método sobrescrito de Cachorro

Animal a2 = new Gato();
a2.fazerSom(); // Executa o método sobrescrito de Gato
```

O tipo da referência (Animal) permite apenas o acesso aos métodos definidos em Animal. Se Cachorro tivesse um método exclusivo chamado abanaRabo(), não poderíamos invocá-lo por meio da referência a1 do tipo Animal.

Chamadas dinâmicas de métodos

Em linguagens como Java, o processo de invocação do método correto em tempo de execução é chamado de dynamic method dispatch. Ele garante que, mesmo usando referências genéricas, o comportamento correto será executado.

Esse mecanismo é fundamental para a implementação de padrões de projeto e APIs extensíveis. Por exemplo:

```
public void emitirSomAnimal(Animal a) {
    a.fazerSom();
}

// Utilização:
emitirSomAnimal(new Cachorro());  // Latido!
emitirSomAnimal(new Gato());     // Miado!
```

Essa função pode receber qualquer objeto que seja uma subclasse de Animal, promovendo o reaproveitamento de código e facilitando futuras extensões.

Benefícios do polimorfismo

Embora sejam inúmeras vantagens do polimorfismo no desenvolvimento de software orientado a objetos, destacamos a seguir alguns. Confira!

Redução do acoplamento

Partes do sistema podem depender de abstrações e não de implementações concretas, facilitando alterações e testes.

Flexibilidade

Objetos de diferentes tipos podem ser tratados de forma genérica.

Reutilização de código

Funções genéricas podem manipular objetos de diferentes classes, desde que compartilhem uma superclasse ou interface.

Extensibilidade

Novas funcionalidades podem ser adicionadas sem alterar o código existente, apenas adicionando novas subclasses.

Considerações sobre o Uso

Apesar de seus benefícios, o polimorfismo precisa ser utilizado com responsabilidade. É importante que:

- O comportamento sobrescrito seja coerente e respeite as expectativas do sistema.
- O uso de referências genéricas não sacrifique a legibilidade ou a clareza do código.
- O contrato estabelecido na superclasse (ou interface) seja cumprido por todas as subclasses.

Por isso, testes e revisões evitam comportamentos inesperados em execuções polimórficas.

Classes abstratas em Java

Descubra neste vídeo como as classes abstratas em Java ajudam a modelar sistemas mais coesos, reutilizáveis e extensíveis. Entenda seu papel na definição de comportamentos obrigatórios, explore exemplos práticos e veja como abstração e polimorfismo se complementam para criar soluções robustas e organizadas no desenvolvimento orientado a objetos.

Conteúdo interativo

Acesse a versão digital para assistir ao vídeo.

Conceito de classes abstratas em Java



Ao construir sistemas complexos, nem sempre faz sentido permitir a instanciação direta de uma classe. Em muitos casos, queremos que uma classe sirva como um modelo conceitual comum para suas subclasses e que apenas essas sejam instanciadas. Para isso, utilizamos as classes abstratas.

As classes abstratas representam conceitos genéricos e incompletos que serão especializados por suas subclasses. Elas podem conter métodos implementados e métodos abstratos (sem corpo), que forçam suas subclasses a fornecerem uma implementação concreta. Esse mecanismo assegura a consistência no design e promove extensibilidade.

Classe abstrata

É declarada com a palavra-chave **abstract** e não pode ser instanciada de maneira direta. Além disso, ela serve como um esqueleto conceitual de um conjunto de entidades relacionadas. Ao contrário das interfaces, classes abstratas podem conter atributos com qualquer nível de visibilidade, métodos concretos (já implementados) e até mesmo construtores. Isso permite definir uma lógica base comum, reutilizável pelas subclasses. Por exemplo:

```java
public abstract class Conta {
    protected String titular;
    protected double saldo;

    public Conta(String titular, double saldo) {
        this.titular = titular;
        this.saldo = saldo;
    }

    public void depositar(double valor) {
        saldo += valor;
    }

    public abstract void sacar(double valor);
}
```

No exemplo, Conta é uma classe abstrata. A operação depositar já possui uma implementação, mas sacar é abstrata: exige que cada tipo específico de conta defina sua própria lógica de saque. Isso é útil quando há

regras diferentes para o saque dependendo do tipo da conta, como conta corrente, poupança ou conta salário.

Subclasses de classes abstratas

As subclasses são obrigadas a implementar todos os métodos abstratos da superclasse, a menos que também sejam declaradas como abstratas. Isso leva qualquer classe concreta a herdar um comportamento completo, funcional. Acompanhe!

```
java

public class ContaCorrente extends Conta {

    public ContaCorrente(String titular, double saldo) {
        super(titular, saldo);
    }

    @Override
    public void sacar(double valor) {
        if (valor <= saldo) {
            saldo -= valor;
        } else {
            System.out.println("Saldo insuficiente.");
        }
    }
}
```

A classe ContaCorrente implementa a lógica do método sacar, que foi apenas declarado na classe abstrata Conta. Com isso, pode-se instanciar ContaCorrente, mas não Conta de maneira direta.



Dica

Uma classe abstrata pode conter tanto métodos abstratos quanto métodos concretos. Essa mescla permite definir um comportamento padrão para algumas operações e delegar outras para as subclasses.

Quando usar classes abstratas

Utilize classes abstratas quando:

- Houver um conceito comum entre múltiplas classes, que deve ser compartilhado.
- Desejar fornecer métodos prontos e, ao mesmo tempo, exigir que métodos específicos sejam obrigatoriamente implementados pelas subclasses.
- Quiser evitar que instâncias sejam criadas a partir de uma classe muito genérica, forçando o uso de versões especializadas.

Além disso, classes abstratas são bastante comuns em frameworks e bibliotecas que definem a estrutura de sistemas maiores. Por exemplo, em um framework MVC, pode-se ter um ControladorAbstrato que define o método processarRequisicao() como abstrato, exigindo que cada controlador específico o implemente conforme a necessidade da aplicação.

Exemplo: sistema de transporte

Vamos considerar um sistema de transporte em que há diversos tipos, mas todos compartilham algumas características e exigem o mesmo tipo de operação, isto é, mover-se. Veja!

```java
public abstract class Transporte {
    protected String modelo;

    public Transporte(String modelo) {
        this.modelo = modelo;
    }

    public abstract void mover();
}

public class Bicicleta extends Transporte {
    public Bicicleta(String modelo) {
        super(modelo);
    }

    @Override
    public void mover() {
        System.out.println("Pedalando a bicicleta " + modelo);
    }
}

public class Aviao extends Transporte {
    public Aviao(String modelo) {
        super(modelo);
    }

    @Override
    public void mover() {
        System.out.println("Voando com o avião " + modelo);
    }
}
```

Uso:

```java
public class TesteTransporte {
    public static void main(String[] args) {
        // Criação dos objetos, exatamente como no exemplo original.
        Transporte t1 = new Bicicleta("Caloi 100");
        Transporte t2 = new Aviao("Boeing 737");

        // Chamada dos métodos, demonstrando o polimorfismo.
        t1.mover(); // Executa o método da classe Bicicleta
        t2.mover(); // Executa o método da classe Aviao
    }
}
```

O exemplo mostra como o polimorfismo e a abstração trabalham em conjunto. O método mover() é invocado a partir de uma variável de tipo Transporte, mas sua execução é determinada pela instância real (Bicicleta ou Avião). Para testar use os passos anteriores.

Vantagens do uso de classes abstratas

Primeiro, vamos estabelecer que classes abstratas são interessantes para a modelagem de sistemas. Elas permitem estruturar seu código de forma clara e robusta, estabelecendo uma hierarquia bem definida e forçando a implementação de comportamentos essenciais. Em sistemas reais, seu uso é comum em frameworks, bibliotecas e arquiteturas baseadas em padrões de projeto. Adotar essa abordagem em seu código resulta em diversas vantagens, como:

- Reduzir a duplicação de código, promovendo a reutilização de lógica comum.
- Garantir a consistência entre as subclasses.
- Facilitar a manutenção do código, centralizando mudanças em comportamentos padrão.
- Permitir a construção de APIs mais robustas, com comportamentos obrigatórios e customizáveis.

Hora de codar

Assista ao vídeo e aprenda a aplicar herança, sobrescrita de métodos e polimorfismo em Java, uma prática essencial para dominar a orientação a objetos.



Conteúdo interativo

Acesse a versão digital para assistir ao vídeo.

Vamos aplicar herança, sobrescrita de métodos e uso do polimorfismo com arrays de referência. Para isso, você deve criar:

1. Uma superclasse Veiculo com os atributos marca e modelo.
2. Duas subclasses: **Carro** (com atributo adicional qtdPortas) e **Moto** (com atributo cilindrada).
3. O método descrever() na superclasse e sobrescrevê-lo nas subclasses com informações específicas.
4. Im array de Veiculo[] com objetos Carro e Moto.

Por fim, itere sobre esse array e chame o método descrever() de forma polimórfica.



Dica

Lembre-se de usar @Override nas subclasses e a palavra-chave super no construtor.

Desafio: nível aventureiro

Assista ao vídeo e evolua sua aventura criando uma hierarquia de personagens. Transforme a classe Personagem em superclasse e implemente Guerreiro e Mago como subclasses com habilidades especiais! Aprenda a utilizar o polimorfismo para armazenar e invocar ações de forma genérica e veja seus heróis ganharem vida em uma jornada orientada a objetos.



Conteúdo interativo

Acesse a versão digital para assistir ao vídeo.

O que você vai fazer

Transformará a classe Personagem (do nível novato) em uma superclasse abstrata. Em seguida, criará subclasses específicas, como Guerreiro e Mago, cada uma com um método usarHabilidadeEspecial() sobrescrito.

Depois, criará uma estrutura (array ou ArrayList) para armazenar diferentes personagens e iterará sobre ela, chamando de forma genérica a habilidade especial de cada um — demonstrando, assim, o poder do polimorfismo de tempo de execução.

Requisitos funcionais

São requisitos funcionais:

- Transformar Personagem em uma classe abstrata.
- Adicionar um método abstrato usarHabilidadeEspecial().
- Criar pelo menos duas subclasses, como Guerreiro e Mago, com atributos e comportamentos próprios.
- Sobrescrever o método usarHabilidadeEspecial() em cada subclasse com uma mensagem personalizada. Exemplos:
    - a) Guerreiro Arthus ataca com Espada Flamejante!
    - b) Mago Elenara lança Bola de Fogo!
- Instanciar diferentes personagens e armazená-los em uma estrutura (Personagem[] ou ArrayList).
- Iterar sobre essa estrutura e chamar o método usarHabilidadeEspecial() para cada personagem.
- Utilizar construtores adequados, usando super() para inicializar os dados da superclasse.
- Criar um método exibirStatus() em Personagem para reaproveitar comportamento comum.

Requisitos não funcionais

São requisitos não funcionais:

- Separar as classes em arquivos distintos (Personagem.java, Guerreiro.java, Mago.java etc.).
- Comentar o código para explicar a lógica e as escolhas.

- Usar convenções de nomeação e boas práticas de identação.
- Atente-se: o programa deve ser executável via terminal ou IDE com saída clara no console.
- Não utilizar estruturas avançadas ainda não abordadas, como herança múltipla, tratamento de exceções ou interfaces neste desafio.

Acessando o desafio

Acesse o GitHub Classroom. Nesse ambiente, você terá acesso ao repositório padrão do desafio. Caso ainda não tenha uma conta no GitHub, não se preocupe: você pode criar uma grátis, clicando no link.

Agrupamento de objetos em Java

Descubra neste vídeo como representar relações entre objetos em Java por meio de associação, agregação e composição. Com exemplos práticos de sistemas acadêmicos e de vendas, você vai aprender a construir estruturas coesas, encapsular corretamente e aplicar boas práticas de modelagem orientada a objetos em cenários reais. Confira!

Conteúdo interativo

Acesse a versão digital para assistir ao vídeo.

Conceito de agrupamento de objetos em Java



Imagine um sistema acadêmico em que cada curso possui uma lista de alunos matriculados — ou um sistema de vendas no qual um pedido agrupa diversos produtos. Situações como essas são comuns no desenvolvimento de sistemas orientados a objetos, e para representá-las de forma clara e eficiente, precisamos compreender o agrupamento de objetos.

Diante disso, vamos entender como representar associações entre classes em Java, sobretudo, do tipo um-para-um e um-para-muitos, explorando as diferenças entre associação, agregação e composição. Também discutiremos boas práticas de encapsulamento e coesão nas relações entre objetos, abordando cenários reais, como sistemas de pedidos, gerenciamento de projetos e catálogos de produtos.

Associação entre objetos

Em Java, podemos representar a relação entre dois objetos quando uma classe possui um atributo que é do tipo de outra classe. Essa relação é chamada de **associação**. Existem diferentes tipos de associação, como um-para-um, um-para-muitos e muitos-para-muitos.

Veja um exemplo simples de associação um-para-um. Vamos considerar uma classe Aluno que possui referência para um objeto da classe Curso:

```java
public class Curso {
    private String nome;

    public Curso(String nome) {
        this.nome = nome;
    }

    public String getNome() {
        return nome;
    }
}

public class Aluno {
    private String nome;
    private Curso curso; // Associação um-para-um

    public Aluno(String nome, Curso curso) {
        this.nome = nome;
        this.curso = curso;
    }

    public String getDados() {
        return nome + " - Curso: " + curso.getNome();
    }
}
```

No exemplo, cada Aluno está associado a um único Curso. Essa associação representa um relacionamento em que um objeto referencia outro.

Associação um-para-muitos

Em muitas aplicações, é comum precisar representar uma coleção de objetos relacionados. Para isso, usamos listas em Java. Considere agora um cenário em que um Curso possui muitos Alunos:

```java
public class Curso {
    private String nome;
    private List alunos;

    public Curso(String nome) {
        this.nome = nome;
        this.alunos = new ArrayList<>();
    }

    public void adicionarAluno(Aluno aluno) {
        alunos.add(aluno);
    }

    public void listarAlunos() {
        for (Aluno a : alunos) {
            System.out.println(a.getDados());
        }
    }
}
```

Agora temos uma associação um-para-muitos, em que um curso contém uma lista de alunos. Esse tipo de estrutura é muito comum em sistemas que envolvem catálogos, registros ou históricos.

Agregação e composição

São formas de associação, mas com diferenças semânticas:

Agregação		Composição
Os objetos associados podem existir de maneira independente.		Os objetos associados dependem da existência do objeto "pai".

Agregação

Um exemplo clássico de agregação é um departamento que contém funcionários que podem ser realocados ou existir fora daquele departamento específico. Veja como seria o código dessas condições:

```
java

public class Funcionario {
    private String nome;

    public Funcionario(String nome) {
        this.nome = nome;
    }

    public String getNome() {
        return nome;
    }
}

public class Departamento {
    private String nome;
    private List funcionarios;

    public Departamento(String nome) {
        this.nome = nome;
        this.funcionarios = new ArrayList<>();
    }

    public void adicionarFuncionario(Funcionario f) {
        funcionarios.add(f);
    }
}
```

Composição

Implica dependência mais forte. Por exemplo, um Pedido contém ItensPedido, e esses itens só existem dentro do contexto do pedido:

```java
public class ItemPedido {
    private String produto;
    private int quantidade;

    public ItemPedido(String produto, int quantidade) {
        this.produto = produto;
        this.quantidade = quantidade;
    }

    public String getDescricao() {
        return quantidade + "x " + produto;
    }
}

public class Pedido {
    private List itens;

    public Pedido() {
        this.itens = new ArrayList<>();
    }

    public void adicionarItem(String produto, int quantidade) {
        ItemPedido item = new ItemPedido(produto, quantidade);
        itens.add(item);
    }

    public void exibirPedido() {
        for (ItemPedido item : itens) {
            System.out.println(item.getDescricao());
        }
    }
}
```

Inicialização segura e encapsulamento

Ao trabalhar com agrupamentos, é importante que a inicialização de listas e objetos associados seja segura. Assim, sempre que possível, inicialize listas no construtor da classe:

```
private List<String> tarefas = new ArrayList<>();
```



Dica

Evite deixar referências nulas que possam gerar NullPointerException. Além disso, utilize modificadores de acesso apropriados (private, protected) e forneça métodos de acesso (getters/setters) quando necessário. Outra prática recomendada é usar interfaces nas declarações de coleções, como List, em vez de classes concretas, como ArrayList. Isso facilita a manutenção e futuras alterações.

Boas práticas no relacionamento entre objetos

Para o bom design no relacionamento entre objetos, siga estas práticas:

- Prefira composições quando os objetos dependem bastante uns dos outros.
- Use agregações para manter o acoplamento baixo.
- Nunca exponha a lista de objetos. Para isso, crie métodos para manipulá-la.
- Aplique o princípio da responsabilidade única: cada classe deve cuidar de um único aspecto.

Agrupando dados com Stream API e Collectors em Java

Aprenda neste vídeo a usar a API de Streams e os coletores da classe Collectors para agrupar dados de forma funcional e eficiente em Java. Com exemplos práticos, explore técnicas como groupingBy, counting e averagingDouble, aplicáveis a sistemas acadêmicos, gestão de tarefas e relatórios corporativos.

> Conteúdo interativo
>
> Acesse a versão digital para assistir ao vídeo.

No desenvolvimento de aplicações modernas — em especial, aquelas que manipulam grandes volumes de dados —, precisamos contar com mecanismos eficientes para organizar e agrupar informações. A API de Streams, introduzida no Java 8, oferece uma abordagem funcional para processamento de coleções, sendo a classe Collectors um dos principais recursos quando o assunto é agrupamento. Vamos explorar como utilizar Collectors.groupingBy() e outras funções relevantes para organizar dados de forma concisa, flexível e legível.

Introdução ao Stream API e Collectors

Vamos, primeiro, entender como funciona a API de Streams. Ela permite realizar operações encadeadas sobre coleções, como filtragem, mapeamento, redução e agrupamento, tornando o código mais legível e menos propenso a erros manuais.

Exemplo simples de uso de stream:

```
java

List nomes = Arrays.asList("Ana", "Bruno", "Carlos", "Ana");
nomes.stream()
     .distinct()
     .sorted()
     .forEach(System.out::println);
```

A classe Collectors provê implementações de coletores que podem ser usados em operações de terminação em streams. Entre os mais importantes estão:

toList()

Coleta elementos em uma lista.

toSet()

Coleta elementos em um conjunto.

joining()

Concatena strings.

groupingBy()

Agrupa elementos com base em uma classificação.

partitioningBy()

Particiona com base em um predicado.

Vale ressaltar que coletores — sobretudo, os vistos aqui — influenciam a construção de pipelines eficientes e poderosos, sendo aplicáveis em diversos domínios do desenvolvimento, como análise de dados, construção de relatórios e modelagem de entidades.

Exemplo 1: Agrupando alunos por curso

Imagine que você está desenvolvendo um sistema para uma universidade e precisa organizar os dados de alunos por curso. Os dados estão em uma lista, e você precisa exibir os alunos de forma agrupada.

```java
class Aluno {
    private String nome;
    private String curso;

    public Aluno(String nome, String curso) {
        this.nome = nome;
        this.curso = curso;
    }

    public String getNome() { return nome; }
    public String getCurso() { return curso; }
}

List alunos = Arrays.asList(
    new Aluno("João", "ADS"),
    new Aluno("Maria", "ADS"),
    new Aluno("Pedro", "Jogos Digitais"),
    new Aluno("Laura", "Redes")
);

Map> agrupadoPorCurso = alunos.stream()
    .collect(Collectors.groupingBy(Aluno::getCurso));

agrupadoPorCurso.forEach((curso, lista) -> {
    System.out.println("Curso: " + curso);
    lista.forEach(a -> System.out.println(" - " + a.getNome()));
});
```

Saída esperada:

```
Curso: ADS
- João
- Maria
Curso: Jogos Digitais
- Pedro
Curso: Redes
- Laura
```

O exemplo mostra como o método groupingBy substitui lógicas tradicionais com if-else e manipulação manual de mapas, trazendo clareza e economia de código.

Exemplo 2: Contando elementos por categoria

Agora vamos supor um sistema de gestão de tarefas. Cada tarefa possui uma prioridade: alta, média ou baixa. Além disso, queremos contar quantas tarefas existem em cada prioridade. Acompanhe!

```
java

class Tarefa {
    private String descricao;
    private String prioridade;

    public Tarefa(String descricao, String prioridade) {
        this.descricao = descricao;
        this.prioridade = prioridade;
    }

    public String getPrioridade() {
        return prioridade;
    }
}

List tarefas = Arrays.asList(
    new Tarefa("Implementar login", "Alta"),
    new Tarefa("Criar banco de dados", "Média"),
    new Tarefa("Testar API", "Alta"),
    new Tarefa("Documentar código", "Baixa")
);

Map contagemPorPrioridade = tarefas.stream()
    .collect(Collectors.groupingBy(
        Tarefa::getPrioridade,
        Collectors.counting()
    ));

contagemPorPrioridade.forEach((prioridade, total) ->
    System.out.println(prioridade + ": " + total));
```

Resultado:

```
Alta: 2

Média: 1

Baixa: 1
```

Esse tipo de agregação é comum em dashboards e painéis administrativos. Assim, entendemos por que essa abordagem é muito útil para análises e monitoramento de tarefas, métricas ou quaisquer entidades categorizadas em um sistema.

Exemplo 3: Agrupando com coleções personalizadas

Você pode escolher a estrutura de coleção que irá armazenar os resultados de um agrupamento. Por exemplo, utilizar Set em vez de List, o que elimina duplicatas de forma automática.

```
java

Map> nomesPorCurso = alunos.stream()
    .collect(Collectors.groupingBy(
        Aluno::getCurso,
        Collectors.mapping(Aluno::getNome, Collectors.toSet())
    ));
```

Essa estrutura mostra como é possível agrupar dados e, ao mesmo tempo, transformar os elementos, criando conjuntos distintos e aplicando múltiplas operações de coleta em uma única linha de código.

Exemplo 4: Média de valores agrupados

Imagine agora um cenário em que você deseja calcular a média salarial dos funcionários por departamento:

```
java

class Funcionario {
    private String nome;
    private String departamento;
    private double salario;

    public Funcionario(String nome, String departamento, double salario) {
        this.nome = nome;
        this.departamento = departamento;
        this.salario = salario;
    }

    public String getDepartamento() { return departamento; }
    public double getSalario() { return salario; }
}

List funcionarios = Arrays.asList(
    new Funcionario("Ana", "TI", 5000),
    new Funcionario("Bruno", "TI", 6000),
    new Funcionario("Carlos", "RH", 4000)
);

Map mediaPorDepartamento = funcionarios.stream()
    .collect(Collectors.groupingBy(
        Funcionario::getDepartamento,
        Collectors.averagingDouble(Funcionario::getSalario)
    ));

mediaPorDepartamento.forEach((dep, media) ->
    System.out.println(dep + ": R$" + media));
```

Saída:

```
TI: R$5500.0

RH: R$4000.0
```

O uso de Collectors é um diferencial importante para qualquer desenvolvedor Java que deseje manipular coleções de maneira performática, elegante e funcional. O agrupamento é uma técnica interessante em sistemas de gestão, relatórios, dashboards e análise de dados. Por fim, o método groupingBy, em especial, deve fazer parte do repertório de qualquer profissional que quer escrever códigos mais declarativos e expressivos.

Coleções em Java

Explore neste vídeo o poder da Java Collections Framework (JCF) para manipular dados com eficiência e clareza. Aprenda a usar List, Set e Map, descubra suas diferenças, vantagens e boas práticas, e veja como integrá-las à API de Streams para operações modernas e performáticas no desenvolvimento de aplicações Java.

Conteúdo interativo

Acesse a versão digital para assistir ao vídeo.

Conceito de coleções em Java

No desenvolvimento de software, é muito comum armazenar, acessar, modificar e percorrer conjuntos de dados. Para isso, o Java oferece um poderoso conjunto de estruturas de dados através do pacote java.util, conhecido como Java Collections Framework (JCF). Vamos agora conhecer mais detalhes!

Motivos para usar coleções



Imagine que você está construindo uma aplicação para uma plataforma de ensino on-line. Cada curso possui uma lista de alunos, uma fila de atendimentos ou mesmo um conjunto de disciplinas. Usar arrays para isso até funcionaria, mas não seria tão flexível, seguro ou produtivo. As coleções permitem abstrair o comportamento de agrupamentos de objetos com mais expressividade, possibilitando ordenação, busca eficiente e duplicidade controlada, entre outros benefícios.

Além disso, ao utilizar coleções, é possível integrá-las com facilidade a APIs modernas da linguagem, como a API de Streams, e aplicar operações complexas de forma declarativa, como filtragens, agrupamentos e mapeamentos, elevando o nível de abstração e produtividade do código.

Estruturas principais da Collections Framework

List

Representa uma coleção ordenada que permite elementos duplicados. A interface mais comum é a ArrayList, baseada em arrays redimensionáveis. Veja!

```
java

import java.util.*;

List linguagens = new ArrayList<>();
linguagens.add("Java");
linguagens.add("Python");
linguagens.add("Java"); // permite duplicatas

System.out.println(linguagens); // [Java, Python, Java]
```

Outras implementações incluem:

LinkedList

Ideal para inserções/remoções frequentes em posições intermediárias.

Vector

Sincronizada, mas, em geral, substituída por ArrayList.

Set

Representa uma coleção que não permite elementos duplicados. A implementação mais usada é o HashSet. Acompanhe!

```java
Set emails = new HashSet<>();
emails.add("ana@email.com");
emails.add("joao@email.com");
emails.add("ana@email.com");

System.out.println(emails); // [ana@email.com, joao@email.com]
```

Outras implementações incluem:

LinkedHashSet

Mantém a ordem de inserção dos elementos.

TreeSet

Preserva os elementos ordenados de forma natural ou por um Comparator fornecido.

Map

Armazena pares chave-valor. A chave é única, e a cada chave um único valor é correspondente. Veja:

```java
Map capital = new HashMap<>();
capital.put("Brasil", "Brasília");
capital.put("França", "Paris");
capital.put("Brasil", "Brasília"); // sobrescreve

System.out.println(capital.get("França")); // Paris
```

Outras opções incluem:

LinkedHashMap

Mantém a ordem de inserção.

TreeMap

Ordena pela chave (natural ou personalizada).

Hashtable

Sincronizado, mas menos usado hoje.

Iterando coleções

a) For tradicional

```java
for (int i = 0; i < linguagens.size(); i++) {
    System.out.println(linguagens.get(i));
}
```

b) For-each

```java
for (String lang : linguagens) {
    System.out.println(lang);
}
```

c) Iterator

```java
Iterator it = linguagens.iterator();
while (it.hasNext()) {
    System.out.println(it.next());
}
```

d) Lambda com forEach

```java
linguagens.forEach(lang -> System.out.println(lang));
```

Integração com Streams

As coleções modernas integram-se com perfeição à API de Streams, o que facilita operações como filtragem, mapeamento e redução.

```java
linguagens.stream()
          .filter(lang -> lang.startsWith("J"))
          .forEach(System.out::println); // Java, Java
```

Streams também permitem agrupamentos, ordenações, contagens, extrações de estatísticas e muito mais, com código legível e eficiente.

Conversões entre coleções

É fácil converter entre listas, conjuntos e mapas:

```java
Set conjunto = new HashSet<>(linguagens);
List novaLista = new ArrayList<>(conjunto);
```

Também é possível converter para arrays:

```java
String[] array = linguagens.toArray(new String[0]);
```

Boas práticas no uso de coleções em Java

Ajudam a ter código mais legível, seguro e eficiente. A seguir, estão algumas recomendações importantes a serem consideradas.

- Escolha apropriada da estrutura: defina se precisa de ordenação, unicidade ou mapeamento.
- Use generics: List<String> em vez de List puro para evitar erros em tempo de execução.
- Evite acessar por índice em List se não for necessário, prefira for-each para legibilidade.
- Não modifique a coleção durante a iteração, use Iterator.remove() se necessário.
- Documente o tipo de coleção esperada em métodos públicos, isso ajuda outros desenvolvedores a usarem sua API de maneira correta.
- Evite classes sincronizadas legadas como Vector ou Hashtable, a menos que a sincronização seja indisénsável e não tratável com Collections.synchronizedX.
- Use interfaces como tipo de variável. Prefira declarar List para facilitar a manutenção, e não ArrayList.

Quando usar cada estrutura?

As coleções em Java manipulam dados de forma estruturada, organizada e performática. O domínio dessas estruturas permite que o desenvolvedor resolva problemas de forma elegante e sem “reinventar a roda”, digamos assim. O uso adequado de List, Set, Map e suas variantes, aliado às boas práticas de iteração e uso de streams, compõe a base para sistemas Java robustos e passíveis de serem mantidos.

A seguir, entenda como definir a estrutura a ser usada para cada requisito.

List (ArrayList)

Manter ordem e permitir duplicatas.

Set (HashSet)

Eliminar duplicatas e ignorar ordem.

Map (HashMap)

Chave associada a valor.

TreeSet / TreeMap

Ordenar elementos automaticamente.

Ordenação em Java: Comparable, Comparator e Streams na prática

Aprenda neste vídeo a ordenar coleções de objetos em Java utilizando as interfaces Comparable e Comparator, além dos recursos modernos da API de Streams. Descubra como organizar dados por critérios personalizados e encadeados, melhorando a legibilidade e eficiência do código em sistemas reais como e-commerces, CRMs e aplicações de relatórios.

Conteúdo interativo

Acesse a versão digital para assistir ao vídeo.

Conceito de ordenação em Java

Ao lidar com coleções de objetos em aplicações Java, é frequente surgir a necessidade de ordená-los conforme critérios específicos, como ordem alfabética, valor numérico ou datas. Vamos entender quais mecanismos a linguagem oferece para realizar essa tarefa: a interface Comparable, a interface Comparator e os recursos integrados da API de Streams para ordenação.

> Por que comparar e ordenar objetos?

Imagine um sistema de gerenciamento de clientes que precisa exibir uma lista de nomes em ordem alfabética ou um e-commerce que ordena os produtos do mais barato para o mais caro. A ordenação de dados torna a informação mais acessível, melhora a experiência do usuário e facilita a implementação de regras de negócio baseadas em critérios quantitativos ou qualitativos.

Interface Comparable<T>

Permite que objetos de uma classe possam ser comparados uns com os outros de forma natural. Ou seja, ela define uma "ordem padrão". Ela ainda exige a implementação do método compareTo().

Exemplo: ordenando livros por título

```java
public class Livro implements Comparable {
    private String titulo;

    public Livro(String titulo) {
        this.titulo = titulo;
    }

    public String getTitulo() {
        return titulo;
    }

    @Override
    public int compareTo(Livro outro) {
        return this.titulo.compareTo(outro.getTitulo());
    }

    @Override
    public String toString() {
        return titulo;
    }
}
```

Uso:

```
List<Livro> livros = new ArrayList<>();

livros.add(new Livro("Java Básico"));

livros.add(new Livro("Algoritmos"));

livros.add(new Livro("Orientação a Objetos"));


Collections.sort(livros);

System.out.println(livros); // [Algoritmos, Java Básico, Orientação a Objetos]
```

Interface Comparator<T>

É usada quando queremos definir múltiplos critérios de ordenação ou quando não podemos alterar a classe do objeto. Essa abordagem é comum em cenários em que objetos precisam ser ordenados de maneiras diferentes, como nome, data ou valor.

Exemplo: ordenando produtos por preço

```
java
public class Produto {
    private String nome;
    private double preco;

    public Produto(String nome, double preco) {
        this.nome = nome;
        this.preco = preco;
    }

    public String getNome() { return nome; }
    public double getPreco() { return preco; }

    @Override
    public String toString() {
        return nome + " - R$ " + preco;
    }
}
Comparator porPreco = new Comparator<>() {
    @Override
    public int compare(Produto p1, Produto p2) {
        return Double.compare(p1.getPreco(), p2.getPreco());
    }
};
```

Uso:

```
List<Produto> produtos = new ArrayList<>();

produtos.add(new Produto("Notebook", 3000));

produtos.add(new Produto("Teclado", 120));

produtos.add(new Produto("Monitor", 900));


Collections.sort(produtos, porPreco);

System.out.println(produtos);

// [Teclado - R$ 120.0, Monitor - R$ 900.0, Notebook - R$ 3000.0]
```

Também é possível usar expressões lambda:

```
produtos.sort((p1, p2) → Double.compare(p1.getPreco(), p2.getPreco()));
```

Ordenação com API de Streams

A API de Streams permite realizar ordenações de forma fluente e declarativa, inclusive com múltiplos critérios encadeados. Confira!

```
java

List ordenados = produtos.stream()
    .sorted(Comparator.comparing(Produto::getNome))
    .collect(Collectors.toList());
```

Ou por múltiplos critérios:

```
java

List ordenadosPorNomeEPreco = produtos.stream()
    .sorted(Comparator.comparing(Produto::getNome)
                    .thenComparing(Produto::getPreco))
    .collect(Collectors.toList());
```

Diferença entre Comparable e Comparator

Uma decisão de design importante é a escolha entre Comparable e Comparator. Enquanto Comparable define uma ordem natural e única para uma classe, Comparator oferece flexibilidade para criar múltiplos critérios de ordenação externos.

Acompanhe a tabela a seguir, que detalha diferenças importantes para ajudar na sua decisão.

Aspecto	Comparable	Comparator
Local de implementação	Dentro da própria classe	Fora da classe
Número de critérios	Apenas um (ordem natural)	Múltiplos (externos e flexíveis)
Flexibilidade	Menor	Maior
Exemplo comum	Ordenar nomes alfabeticamente	Ordenar por data, preço etc.

Tabela: Comparativo entre as interfaces Comparable e Comparator.
Curadoria de TI.

Boas práticas na ordenação

A capacidade de comparar e ordenar objetos é uma competência básica para qualquer programador Java que queira construir sistemas escaláveis, intuitivos e alinhados às expectativas do mercado. Dominar as interfaces Comparable e Comparator, assim como aplicar de maneira certa os recursos da API de Streams, permite organizar dados com fluidez, adaptando-se a diferentes contextos e requisitos de negócio. Ao aplicar esses conceitos, o desenvolvedor agrega valor ao projeto, à equipe e à sua própria formação profissional.

Para que sua lógica de ordenação seja eficiente, clara e manutenível, siga estas boas práticas:

- Sempre que possível, use Comparator.comparing com métodos de referência para tornar o código legível.
- Use thenComparing para ordenação secundária.

- Em classes com muitos critérios, considere separar os comparadores em classes ou métodos auxiliares.
- Documente o critério de ordenação se ele não for óbvio.
- Evite lógica de comparação ambígua, que pode gerar inconsistência na ordenação.

Hora de codar

Assista ao vídeo e aprenda a utilizar associações, listas e ordenação em Java.

Conteúdo interativo

Acesse a versão digital para assistir ao vídeo.

Vamos praticar? Utilize associações, ArrayList, e ordenação com Comparable. Para isso:

1. Crie uma classe Aluno com os atributos nome (String) e notaFinal (double).
2. Implemente o método toString() e a interface Comparable para ordenar por notaFinal.
3. Crie um ArrayList<Aluno>, adicione, pelo menos, 5 alunos com notas diferentes.
4. Ordene a lista usando Collections.sort() e imprima o resultado.

Dica

Implemente o método compareTo(Aluno outro) comparando as notas finais com Double.compare().

Desafio: nível mestre

Assista ao vídeo, monte um grupo de personagens e leve-os ao combate em duelos épicos!

Conteúdo interativo

Acesse a versão digital para assistir ao vídeo.

O que você vai fazer

Você criará uma classe chamada Grupo, que representa um conjunto de personagens. A classe vai ter uma lista de Personagem (já criada nos níveis anteriores) e métodos para adicionar membros ao grupo, exibir informações e realizar batalhas.

Em seguida, você criará um método batalhar(Personagem a, Personagem b) que calcula o poder total de cada personagem com base na fórmula:

$$\text{poderTotal} = \text{nivel} * \text{poderBase};$$

O personagem com o maior poder total vence a batalha. A saída deve indicar qual personagem venceu e qual foi o cálculo do seu poder.

Para expandir ainda mais o desafio, será possível criar uma Arena capaz de gerenciar múltiplas batalhas entre dois grupos. Você também poderá implementar a interface Comparable para permitir a ordenação de personagens por nível antes das batalhas.

Requisitos funcionais

São requisitos funcionais:

- Criar a classe Grupo com um atributo:

  private ArrayList<Personagem> membros;

- Implementar os seguintes métodos:
  - adicionarPersonagem(Personagem p)
  - listarPersonagens()
  - batalhar(Personagem a, Personagem b): realiza a batalha e exibe o vencedor
- Aplicar a fórmula:

  int poderTotal = personagem.getNivel() * personagem.getPoderBase();

- Mostrar o resultado da batalha com:

  Mago Elenara venceu! Poder total: 300

- Criar múltiplos personagens e adicioná-los a dois grupos distintos.
- Extra (opcional): criar a classe Arena com:
  - batalharGrupos(Grupo g1, Grupo g2): realiza batalhas entre os membros dos dois grupos, em pares.
- Extra (opcional): implementar Comparable com base no nível e ordenar os personagens de cada grupo antes das batalhas.

Requisitos não funcionais

São requisitos não funcionais:

- Organizar cada classe em um arquivo separado.
- Utilizar boas práticas de programação: nomes significativos, identação e encapsulamento.
- Comentar trechos relevantes para explicação da lógica.
- Utilizar listas com generics de forma correta (ArrayList).
- Evitar lógica repetida, preferindo métodos reutilizáveis.

Acessando o desafio

Acesse o GitHub Classroom. Nesse ambiente, você terá acesso ao repositório padrão do desafio. Caso ainda não tenha uma conta no GitHub, não se preocupe: você pode criar uma grátis, clicando no link.

Considerações finais

Parabéns pela conquista!

Felicitações por concluir a jornada em Os guardiões de Eldoria! Você enfrentou três etapas desafiadoras, construindo uma base sólida em programação orientada a objetos com Java. Sua aventura começou com a criação de classes e objetos, evoluiu para a especialização de heróis com herança e polimorfismo e culminou no gerenciamento de grupos e batalhas usando coleções.

Ao longo do caminho, você aprendeu a pensar de forma orientada a objetos: modelando entidades, encapsulando dados e criando sistemas flexíveis. Você viu na prática como cada conceito, de uma simples classe Personagem a uma List<Personagem>, é um bloco de construção para dar vida a um mundo digital.

Esse percurso reforçou suas habilidades e sua capacidade de transformar problemas em soluções de software organizadas e escaláveis. Mais do que códigos, você desenvolveu senso crítico sobre design, coesão e as boas práticas que o mercado valoriza. Continue praticando!

Entregando seu projeto

Para concluir o desafio, é necessário que você faça o commit no GitHub do que você desenvolveu. Certifique-se de organizar e documentar de maneira adequada seu repositório. Essa etapa ajuda a demonstrar seu aprendizado.

Por fim, cadastre na Guia de trabalhos da SAVA o link do seu repositório.

Atenção! Caso você atualize o seu repositório, não é necessário alterá-lo na SAVA.

Continue praticando

A melhor forma de solidificar seu conhecimento em POO é aplicá-lo em novos projetos. Aqui estão algumas ideias para desenvolver o que você aprendeu:

- **Sistema de inventário:** crie uma classe Item abstrata e subclasses como Pocao, Arma e Armadura. Faça com que a classe Personagem tenha um inventário (List<Item>), permitindo adicionar, remover e usar itens.
- **Implementar interfaces:** pesquise sobre interface em Java. Crie uma interface Combativel com um método atacar() e faça com que Personagem, e talvez uma nova classe Monstro, implementem-na. Isso levará seu entendimento de polimorfismo a outro nível.
- **Cadastro de clientes e produtos:** crie um sistema simples para uma loja virtual. Modele as classes Cliente, Produto e Pedido, em que um Pedido contém um Cliente e uma List<Produto>. Use Map para buscar produtos por um código único.

Referências

BLOCH, J. **Java Efetivo.** 3. ed. Rio de Janeiro: Alta Books, 2019.

DEITEL, P.; DEITEL, H. **Java:** como programar**.** 10. ed. São Paulo: Pearson Prentice Hall, 2016.

MARTIN, R. C. **Código Limpo:** Habilidades Práticas do Agile Software. Rio de Janeiro: Alta Books, 2009.

ORACLE. **The Java™ Tutorials – The Collections Framework.** 2024.

SIERRA, K.; BATES, B. **Use a cabeça**! **Java**. 2. ed. Rio de Janeiro: Alta Books, 2007.